TouchSynth

An MCU-controlled analog monophonic synthesizer
on a single 4-layer board

Design report, revision 0.1

cpldcpu Claude Opus 5.5

27 September 2026

Abstract

TouchSynth is a small monophonic analog synthesizer that is played on a capacitive touch keyboard and controlled entirely by a CH32V003/CH32V002 microcontroller. It has no potentiometers or mechanical switches. The sound path is fully analog:

- a timer-reset integrator oscillator (DCO),
- a four-pole transistor ladder filter with voltage-controlled resonance,
- a BJT differential-pair VCA with a firmware-calibrated offset trim.

All control voltages are generated by PWM and passive filters. The board uses only JLCPCB *Basic* and *Promotional Extended* parts; the MCU is hand-soldered from own stock.

This report documents the requirements, the circuit design and its justification by ngspice simulation (seven blocks plus the complete voice), a behavioural audio model, the 4-layer PCB, the bill of materials and the verification and validation plan for the boards now on order.

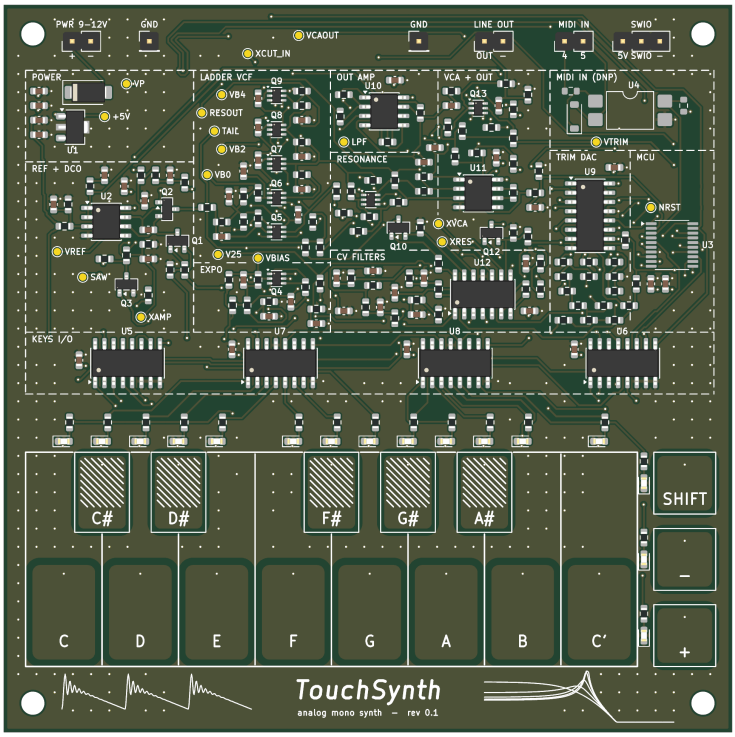


Figure 1: The board as ordered (rendered in KiCad 10), 100 × 100 mm, 4 layers.

Contents

1 Introduction **4**
1.1 Goal and constraints 4
1.2 Tool flow 4

2 Requirements **5**

3 System architecture **5**
3.1 Control concept 6
3.2 MCU pin assignment 6

4 Circuit design **6**
4.1 Power and references 6
4.2 Oscillator (DCO) 7
4.3 Control-voltage filters 7
4.4 Ladder filter, exponential converter and output amplifier 8
4.5 Resonance 8
4.6 VCA, trim DAC and output 8
4.7 Touch keyboard, LEDs and MIDI 8

5 Simulation **8**
5.1 Method 8
5.2 S1 – Oscillator 9
5.3 S2 – Control-voltage filters 9
5.4 S3 – Ladder filter 10
5.5 S4 – VCA 10
5.6 S5 – Touch sensing 10
5.7 S6 – Complete voice 11
5.8 S7 – Power and references 11
5.9 Design changes found by simulation 12

6 Behavioural model and audio **12**

7 PCB design **13**
7.1 Board and stack-up 13
7.2 Floorplan 14
7.3 Placement 15
7.4 Routing 16
7.5 Silkscreen 17
7.6 Fabrication data 17

8 Bill of materials **17**

9 Firmware **19**

10 Verification and validation plan **21**
10.1 Bring-up 22
10.2 Functional tests 22
10.3 Calibration and validation 23

11 Open points and risks **23**

A Schematic **24**

B Reproducing the design **33**

1 Introduction

1.1 Goal and constraints

The project had three aims:

1. Design a complete, self-contained electronic instrument end to end: requirements, circuit design, simulation, PCB layout and fabrication.
2. Keep the part count and the cost of assembly low.
3. Build in enough observability that the board can be verified automatically once it is back from the factory.

The following constraints shaped the design:

- **Parts**: only JLCPCB *Basic* and *Promotional Extended* parts, so there are no feeder loading fees and the board can be assembled in JLCPCB's economic PCBA service. The part lists were downloaded from the JLCPCB parts API (351 Basic and 1235 Promotional parts) and every BOM line was chosen from them.
- **MCU**: a CH32V003F4P6 or CH32V002F4P6 in TSSOP-20 from own stock, hand-soldered (do-not-populate in the assembly BOM). Both variants are pin compatible; one firmware supports both.
- **User interface**: no potentiometers and no mechanical switches. Thirteen capacitive keys (one octave C–C) plus three function pads (SHIFT, −, +), and 16 LEDs.
- **Power**: 9 . . . 12 V DC on a 2-pin header, reverse-polarity protected.
- **Scope**: a demonstrator. Some imperfections (power-on pop, filter temperature drift) are accepted explicitly rather than solved with additional parts. MIDI input is laid out but not populated.

1.2 Tool flow

The design was carried out with open tools and scripts, so every artefact can be regenerated (Table 1). A single Python description of the circuit (`hw/design.py`) is the source of truth: the schematic, the PCB netlist and the simulation cross-check are all derived from it.

Table 1: Tool flow and where each step lives in the repository.

Step	Tool	Files
Part selection	JLCPCB parts API	`parts/*.csv`
Circuit simulation	ngspice-42 (WSL), Python	`sim/s1_dco` … `sim/s7_power`, `sim/lib/voice.py`
Behavioural audio model	Python + numba	`sim/behav/`
Circuit description	Python	`hw/design.py` (242 parts, 166 nets)
Schematic	generated KiCad 10 schematic	`gen_sch2.py`, `layouts.py`, `draw.py` (in `hw/`)
Checks	kicad-cli ERC, netlist and simulation cross-check	`check_netlist.py`, `check_sim.py` (in `hw/`)
PCB	KiCad 10 Python API, Freerouting 1.9, own maze router	`hw/gen_pcb.py`, `hw/pcb_*.py`
Fabrication data	kicad-cli	`hw/fab.py`
Firmware	ch32fun (git submodule), riscv64-unknown-elf-gcc	`firmware/`

2 Requirements

Table 2 summarises the functional requirements (full text in `docs/spec.md`). Each requirement is traced to a simulation and to a hardware test in Section 10.

Table 2: Functional requirements.

Block	Requirement
DCO	Pitch from an MCU timer, range $\geq$ C1 (32.7 Hz) … C7 (2093 Hz); saw amplitude constant within $\pm 5\,\%$ over the range
Waveforms	saw, pulse with variable width, sub-oscillator (-1 octave square), noise; each switchable
VCF	24 dB/oct transistor ladder; cutoff 30 … 15 000 Hz via a hardware exponential converter; resonance from zero to self-oscillation
VCA	≥ 70 dB attenuation when closed; envelope from firmware; CV feed-through nulled by a calibrated trim
Output	line level $\approx 1\,V_{pp}$ into $\geq 10\,k\Omega$, AC coupled
Control voltages	4 PWM channels, 12-bit effective resolution; ripple < -70 dB of full scale; settling $<$ 2 ms
Touch	16 pads; full scan ≥ 200 Hz; touch signal $\geq 5\times$ noise
Power	9 … 12 V, reverse protected, ≤ 60 mA
Test	audio loopback to the MCU ADC; test points; programming/debug over SWIO

3 System architecture

Figure 2 shows the architecture:

- **Audio path** (top row): analog from the oscillator to the line output.
- **MCU**: generates the oscillator timing and the waveform bits directly. The four continuous parameters (oscillator amplitude, cutoff, resonance, VCA level) are PWM outputs of TIM1, smoothed by passive three-pole filters and converted to currents.
- **Touch pads**: read through two CD4051 multiplexers with the ADC charge-slope method.
- **Shift-register chain**: three 74HC595 drive the 16 LEDs and an 8-bit R-2R DAC that trims the VCA offset.
- **Loopback**: a divided copy of the output returns to an ADC pin. It is used for self-test and calibration.

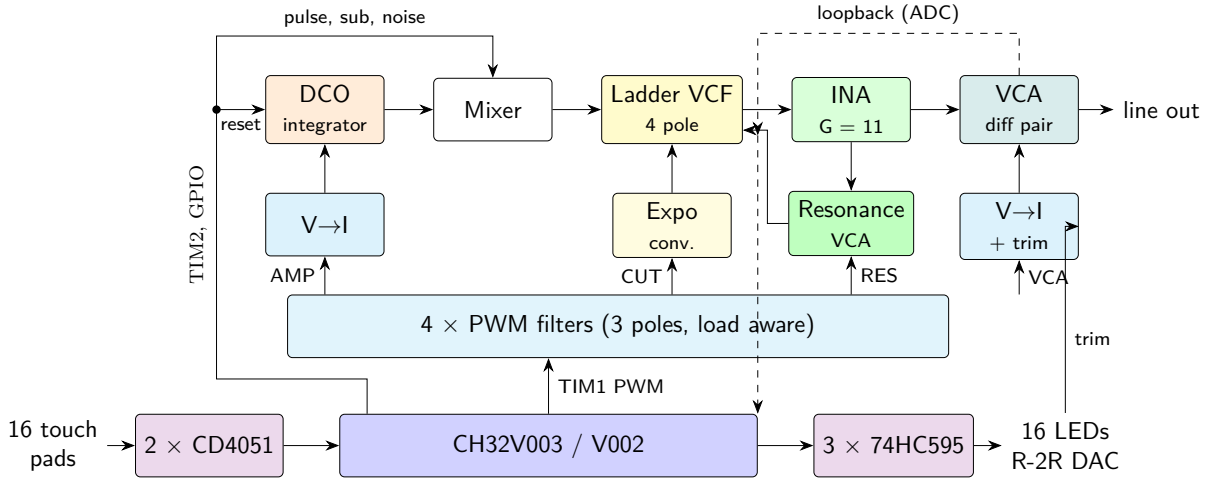


Figure 2: System architecture. The audio path is analog; the MCU sets all parameters through PWM control voltages and timing, and observes the output through the loopback.

3.1 Control concept

- **Oscillator timing**: TIM2 CH1 produces one short reset pulse per period, so the pitch is as exact as the 48 MHz timer. The saw amplitude is kept constant by making the integrator charging current proportional to the frequency (CV_{AMP}).
- **Waveform bits**: the pulse wave comes from TIM2 CH2. The sub-oscillator is a GPIO toggled in the TIM2 update interrupt, and noise is a GPIO driven by a firmware LFSR.
- **Control voltages**: TIM1 CH1–CH4 run at 10 bit and 46.9 kHz. The compare registers are updated by DMA bursts with 2-bit sigma-delta dithering, which gives 12-bit effective resolution at a high carrier frequency.
- **Firmware control loop**: envelopes, LFO, glide and the arpeggiator run in firmware at a 1 kHz control rate.

3.2 MCU pin assignment

Table 3 lists the pin map. The LED/DAC shift registers share the DATA and CLOCK lines with the multiplexer select lines S0 and S1, so no third multiplexer is needed.

Table 3: MCU pin map (TSSOP-20, identical for CH32V003 and CH32V002).

Pin	Port	Function	Pin	Port	Function
1	PD4	74HC595 LATCH	11	PC1	SUB oscillator (GPIO)
2	PD5	ADC A5 ← audio loopback	12	PC2	TIM2 CH2 → PULSE
3	PD6	USART RX ← MIDI	13	PC3	mux S0 / 595 DATA
4	PD7	NRST	14	PC4	mux S1 / 595 CLK
5	PA1	touch mux B → ADC A1	15	PC5	TIM2 CH1 → DCO reset
6	PA2	touch mux A → ADC A0	16	PC6	TIM1 CH1 → CV_{AMP}
7	VSS	GND	17	PC7	TIM1 CH2 → CV_{CUT}
8	PD0	NOISE (GPIO)	18	PD1	SWIO
9	VDD	+5 V	19	PD2	mux S2
10	PC0	TIM1 CH3 → CV_{RES}	20	PD3	TIM1 CH4 → CV_{VCA}

4 Circuit design

The complete schematic (7 hierarchical sheets) is reproduced in Appendix A. This section explains each block and its key values. Unless noted, all resistors are 1 % thick film 0603, and all capacitors are X7R/X5R 0603.

4.1 Power and references

- **Input**: an SS14 Schottky diode protects against reverse polarity. The protected rail V+ (8.6 . . . 11.6 V) supplies the analog circuitry.
- **+5 V**: a 78L05 supplies the MCU, the multiplexers, the shift registers and the LEDs.
- **Audio reference**: VREF = 4.5 V is derived from +5 V (1 kΩ/9.1 kΩ divider, 10 µF) and buffered by a TL072 follower. The follower has a 10 Ω + 1 µF output snubber instead of a bare capacitor. The simulation showed that the snubber is required for stability, see Section 5.
- **Other references**: VBIAS = 1.5 V and V25 = 2.5 V are plain dividers from +5 V. Referencing everything that sets pitch or cutoff to +5 V, not to V+, makes the sound independent of the input voltage.

4.2 Oscillator (DCO)

The oscillator is a TL072 inverting integrator ($C_{\text{int}} = 10\,\text{nF}$) whose non-inverting input sits at VREF (Figure 3):

- **Charging**: a V→I converter (LM324 + MMBT3904, $R_E = 33\,\text{k}\Omega$) draws the charging current from the summing node, so the saw ramps up from 4.5 V.
- **Reset**: once per period a 3 µs pulse from TIM2 CH1 goes through an MMBT3904 level shifter to a 2N7002 across the capacitor. A 2.2 kΩ gate resistor softens the charge-injection spike.
- **Constant amplitude**: the firmware sets $I = A\,C_{\text{int}}/(T - t_{\text{reset}})$, so the saw amplitude A does not depend on the period T.

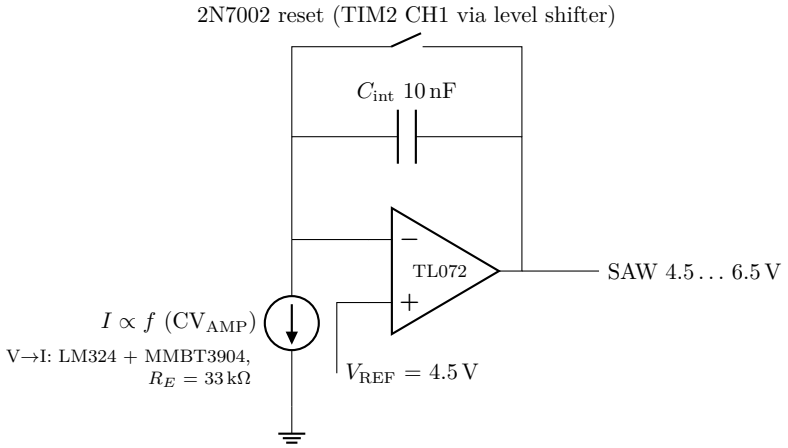


Figure 3: DCO core (simplified): integrator with current-sink charging and a MOSFET reset switch.

Mixer. The saw (150 kΩ), pulse, sub and noise (470 kΩ each) are summed into 10 kΩ to the ladder input bias VB0 through 1 µF.

4.3 Control-voltage filters

Each TIM1 PWM output is smoothed by three RC poles (Figure 4):

- **LIN networks** (CV$_{\text{AMP}}$, CV$_{\text{RES}}$, CV$_{\text{VCA}}$): the last stage is a $4.7\,\text{k}\Omega/15\,\text{k}\Omega \parallel 22\,\text{nF}$ divider into an LM324 V→I converter. The DC ratio is 0.591.
- **EXP network** (CV$_{\text{CUT}}$): the last stage is a $33\,\text{k}\Omega/1.5\,\text{k}\Omega \parallel 100\,\text{nF}$ divider to VBIAS. It drives the exponential converter base directly, with a ratio of 0.0373 (10.4 octaves over the 5 V PWM range).

Both ratios include the series resistance of the first two stages; Section 5.9 describes the bug that made this explicit.

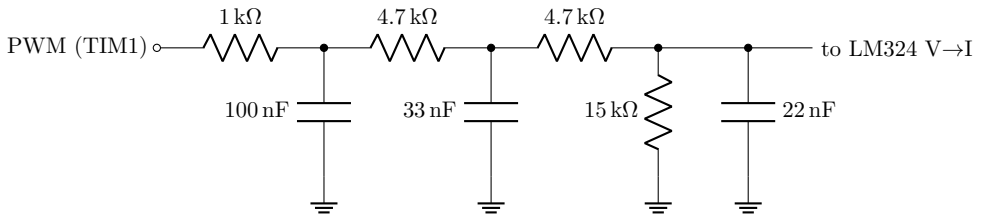


Figure 4: LIN control-voltage network. The EXP variant (CV$_{\text{CUT}}$) replaces the last stage by $33\,\text{k}\Omega$ / $1.5\,\text{k}\Omega \parallel 100\,\text{nF}$ to VBIAS.

4.4 Ladder filter, exponential converter and output amplifier

- **Ladder**: a Moog-type ladder made of five BC847BS dual NPN transistors (input pair, three rungs, load pair) and four 33 nF capacitors.
- **Bias chain**: a divider from V+ (2.2 kΩ, 4×1 kΩ, 2.4 kΩ, each tap decoupled with 10 µF) sets the base voltages VB0 … VB4.
- **Exponential converter**: the tail current comes from a BC847BS pair with an LM324. The reference current is 8.3 µA through 300 kΩ, and the loop is compensated with 1 nF.
- **Output amplifier**: the differential output across the top capacitor is read by a TL072 two-op-amp instrumentation amplifier (gain 11, output referenced to VREF). Its JFET inputs do not load the ladder, whose source impedance reaches about 50 kΩ at low cutoff.

4.5 Resonance

The resonance loop is a voltage-controlled amplifier of its own:

- **Input**: the filter output is attenuated 1:13 and AC coupled into a BC847BS differential pair.
- **Gain control**: the pair's tail current comes from CV_{RES} through an LM324 V→I converter ($R_E = 6.2\,k\Omega$).
- **Feedback**: the collectors feed a TL072 unity difference amplifier. Its output returns through 4.7 µF and 22 kΩ into the second input of the ladder.

The difference amplifier removes the common-mode step of the collectors when the tail current changes. An earlier single-ended version injected volt-level DC steps into the ladder.

4.6 VCA, trim DAC and output

- **Core**: a BC847BS differential pair (bases biased from VREF, input attenuated 1:34) with 4.7 kΩ collector resistors. A TL072 difference amplifier (10 kΩ/22 kΩ) follows, then 10 µF to the line output.
- **Tail current**: an LM324 V→I converter ($R_E = 4.7\,k\Omega$) supplies it. A 470 kΩ resistor from +5 V to the emitter makes the current truly zero at CV = 0 despite the op-amp offset voltage.
- **Offset trim**: the third 74HC595 in the LED chain drives an 8-bit R-2R DAC (100 kΩ/200 kΩ). The DAC injects a small offset current into one base (1 MΩ, with 2 MΩ fixed on the other base). The firmware finds the DAC code that nulls the CV feed-through ("thump").
- **Loopback**: a 10 kΩ/15 kΩ divider with 1 nF feeds the output to PD5 (ADC A5).

4.7 Touch keyboard, LEDs and MIDI

- **Touch pads**: the 16 pads are copper under solder mask (white keys 8.6 mm × 13 mm, black keys 6.5 mm × 10 mm, function pads 7.5 mm square). Each group of eight goes through a CD4051 to one ADC channel.
- **Measurement**: the ch32fun ADC slope method (`extralibs/ch32v00x_touch.h`) measures each pad.
- **LEDs**: each of the 16 red 0603 LEDs has its own 4.7 kΩ series resistor (about 0.6 mA).
- **MIDI**: an LTV-817 optocoupler feeds the USART RX pin. These parts are not populated; the footprints are kept.

5 Simulation

5.1 Method

- **Simulator**: every block was simulated in ngspice-42, driven by Python scripts. The shared netlist fragments are in `sim/lib/voice.py`, so all simulations use identical circuits.

- **Models**: TI Boyle macromodels for the TL072 and LM324, NXP Gummel-Poon models for the BC847B, generic MMBT3904 and 2N7002 models, and a behavioural 78L05.
- **Supply corners**: V+ = 8.6 V and 11.6 V.
- **Firmware in the loop**: the firmware is modelled where it matters. Timer periods are quantised at 48 MHz, the CVs are 12-bit codes, and the control rate is 1 kHz.
- **Calibration data**: calibrations found by one simulation (DCO gain, cutoff table, VCA law, resonance threshold) are written to CSV files and read back by the others, never copied by hand.
- **Cross-check**: `hw/check_sim.py` verifies that the 145 simulated components match the schematic values.

Table 4 summarises the results. All blocks meet their requirements.

Table 4: Simulation summary.

#	Block	Requirement	Result	Status
S1	DCO	amplitude flat ±5 %, C1–C8, 9 . . . 12 V	−2.2 / +0.8 %, supply independent	pass
S2	CV filters	ripple < −70 dB FS, settling < 2 ms	−81 / −84 dB; 1.3 / 1.2 ms (1 %)	pass
S3	Ladder VCF	30 . . . 15 000 Hz, 24 dB/oct, self-oscillation	19.9 . . . 19 800 Hz, −23 dB/oct, self-osc. from $CV_{RES} \approx 3.1$ V	pass
S4	VCA	≥ 70 dB off, low feed-through	121 dB; thump 94–101 mV → < 0.6 mV after trim	pass
S5	Touch	Δ ≥ 5× noise, ≥ 200 Hz	SNR 39 (V003) / 152 (V002) worst case; 1.4 kHz scan	pass
S6	Full voice	plausible sound, loopback in range	1.22 V_{pp} line out; loopback 2.3 . . . 2.9 V	pass
S7	Power	9 . . . 12 V, ≤ 60 mA, low hum	33 / 42 mA; hum −81 dB	pass

5.2 S1 – Oscillator

Figure 5 shows the saw waveforms and the reset. Figure 6 shows the amplitude versus the note.

- **Ramp**: rises from 4.53 V to 6.50 V. The −2.2 % gain error is removed by one calibration constant.
- **Flatness**: over C1–C8 and both supply corners it stays within −2.2 %/0.8 %. The low end is limited by the 12-bit quantisation and the LM324 offset.
- **Reset**: completes in about 2.5 µs. Charge injection leaves a 0.43 V, 100 ns spike, which is negligible against the 2 V reset step.

5.3 S2 – Control-voltage filters

The first iteration used 12-bit PWM at 11.7 kHz. It reached only −66.7 dB ripple and 2.2 ms settling. Switching to 10-bit PWM at 46.9 kHz with 2-bit sigma-delta dithering keeps 12-bit resolution and moves the carrier up by a factor of four.

Table 5: S2 results at the worst-case duty cycle (50 %), dither active.

Channel	Full scale	Ripple	re FS	Settling 1 %	Settling 0.1 %
LIN	2.95 V	271 μV_{pp}	−81 dB	1.31 ms	1.88 ms
EXP	187 mV	12 μV_{pp}	−84 dB	1.22 ms	1.68 ms

The EXP ripple corresponds to 0.8 cent of cutoff modulation.

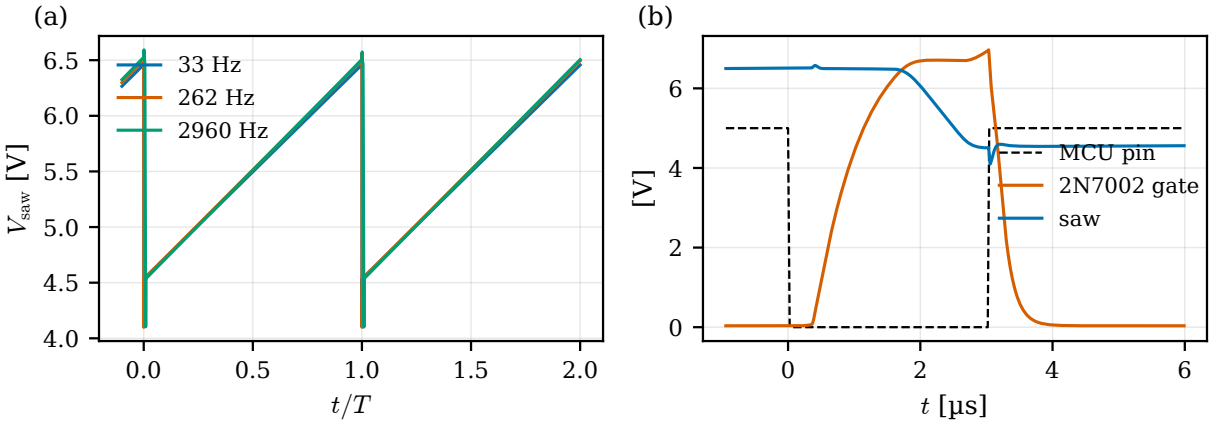


Figure 5: S1: (a) saw at 33 Hz, 262 Hz and 2.96 kHz over two periods (time normalised to the period); (b) reset detail at 4.19 kHz: MCU pin, 2N7002 gate and saw.

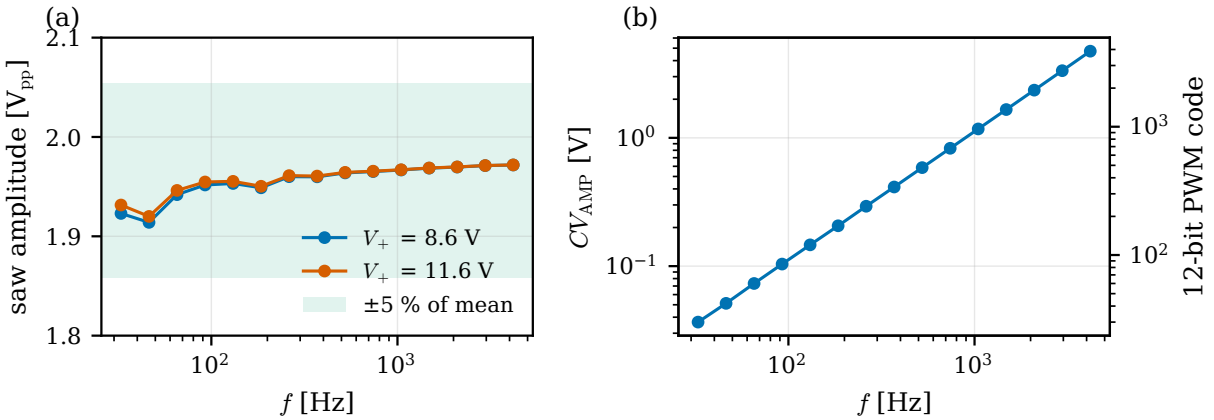


Figure 6: S1: (a) saw amplitude versus note at both supply corners; (b) firmware CV_{AMP} law and 12-bit code.

5.4 S3 – Ladder filter

5.5 S4 – VCA

- **Gain**: maximum 7.4 dB. THD is 0.06 / 0.21 / 0.81 % at 0.3 / 0.6 / 1.2 V_{pp} input.
- **Off attenuation**: 121 dB. Without the 470 kΩ anti-offset resistor a 7 mV LM324 offset would leave only 52 dB.
- **Feed-through**: with realistic mismatch (±1 % collector resistors, 1 . . . 2 mV V_{BE} offset) the CV feed-through is 94–101 mV per note, an audible thump. Both error terms scale with the tail current, so one static offset trim cancels both. After the two-point calibration the residual is 0.58 mV and 0.08 mV in the two mismatch cases.

5.6 S5 – Touch sensing

- **Model**: the ch32fun slope method. The pin is released to the internal pull-down, and the ADC sample-and-hold samples the first 0.625 µs of the decay. The model includes the multiplexer (30 pF common capacitance, 470 Ω on-resistance), the trace and the pad.
- **Sensitivity**: the multiplexer costs about a factor of five in sensitivity compared with a direct pin, but the margin stays large. A 5 pF touch gives 14.8–20.8 LSB per sample (V003) or 57–81 LSB (V002). The worst-case SNR is 39 and 152 with 16 samples per pad.
- **Scan rate**: a full scan takes 0.69 ms (1.4 kHz).

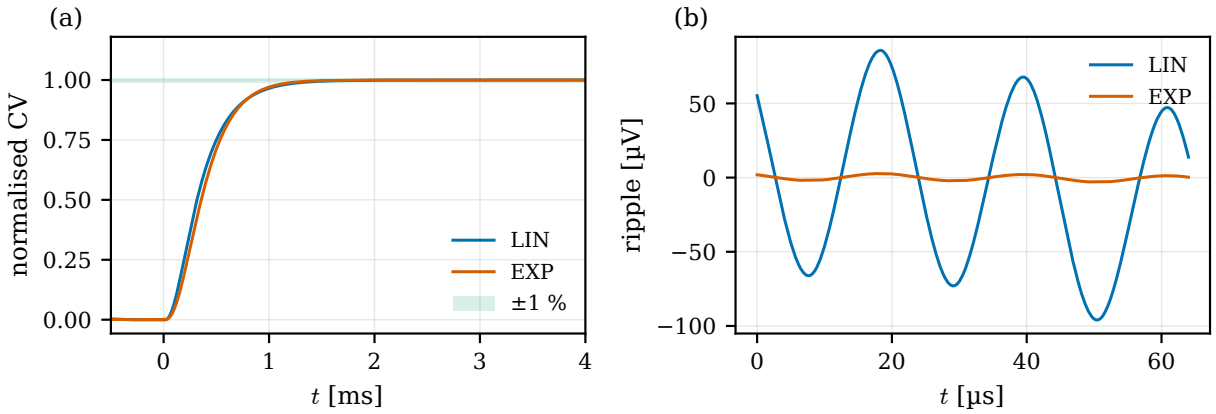


Figure 7: S2: (a) normalised 10 . . . 90 % step response; (b) steady-state ripple at 50 % duty with dither.

Table 6: S3 results.

Quantity	Value
−3 dB cutoff range (CV_{CUT} 0–5 V)	19.9 . . . 19 800 Hz (≈ 10 octaves)
Tracking	2.05 oct/V, log-linear
Slope (one octave above cutoff)	−23.1 dB/oct
Passband gain (ladder in → INA out)	20.4 dB, flat within 1.6 dB
Resonance peak at CV_{CUT} = 2.5 V	4.8 / 13.6 / 20.7 / 28.9 dB at CV_{RES} = 1 / 2 / 2.5 / 2.8 V
Self-oscillation threshold	CV_{RES} ≈ 3.1 V; 0.11 . . . 0.29 V_{pp} (soft-limited)
Supply sensitivity	+19 cent between 9 V and 12 V input
Temperature drift	−13.4 cent/K (accepted, see Section 11)

5.7 S6 – Complete voice

The complete analog circuit was simulated with the firmware behaviour modelled in Python:

- quantised timer periods,
- a sub-oscillator toggled once per period,
- a pulse-width LFO,
- ADSR envelopes,
- all CVs mapped through the calibrations of S1, S3 and S4.

The score is an eight-note bass line with filter envelope, followed by a held note with a slow resonant sweep. The line output peaks at 1.22 V_{pp}. The loopback node stays within 2.34 . . . 2.88 V, well inside the ADC range.

5.8 S7 – Power and references

- **Ripple**: with 200 mV_{pp} of 100 Hz ripple on the input, the line-output hum is −81 dB relative to 1.33 V_{pp} at both 9 V and 12 V.
- **Current**: 33 and 42 mA, including an assumed 15 mA for the logic and LEDs.
- **VREF buffer history**: the first VREF buffer (TL072 driving 10 µF through 100 Ω outside the loop) let 1.1 mV_{pp} of hum through, −47 dB at the output. A dual-loop variant fixed the hum but had a 48 Ω impedance peak. The final plain follower with an RC snubber has 5 mΩ at 100 Hz and a single well-damped peak.
- **Power-on pop**: about 3 V, decaying in about 0.3 s. It is accepted: switch the synth on before the amplifier.

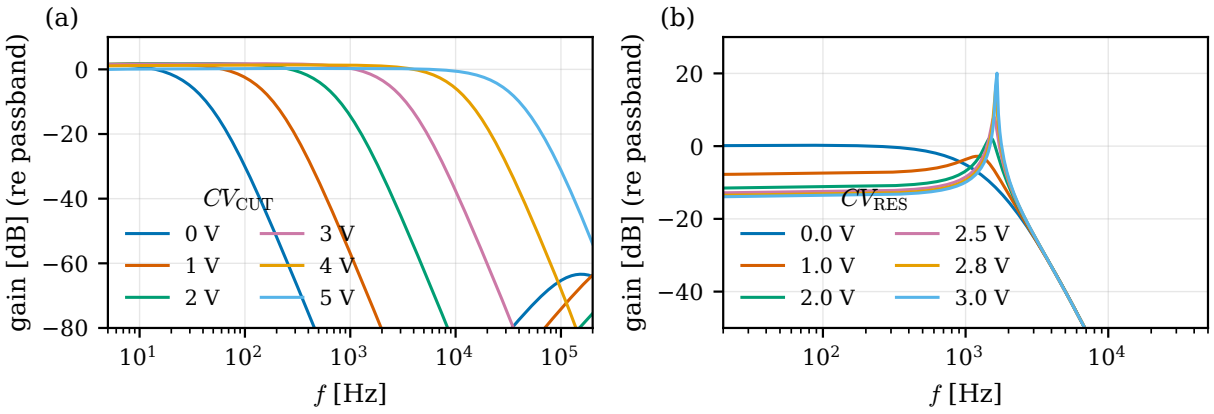


Figure 8: S3: (a) filter response for $\mathrm{CV_{CUT}} = 0$–$5\,\mathrm{V}$ without resonance; (b) response at $\mathrm{CV_{CUT}} = 2.5\,\mathrm{V}$ for $\mathrm{CV_{RES}}$ below the self-oscillation threshold.

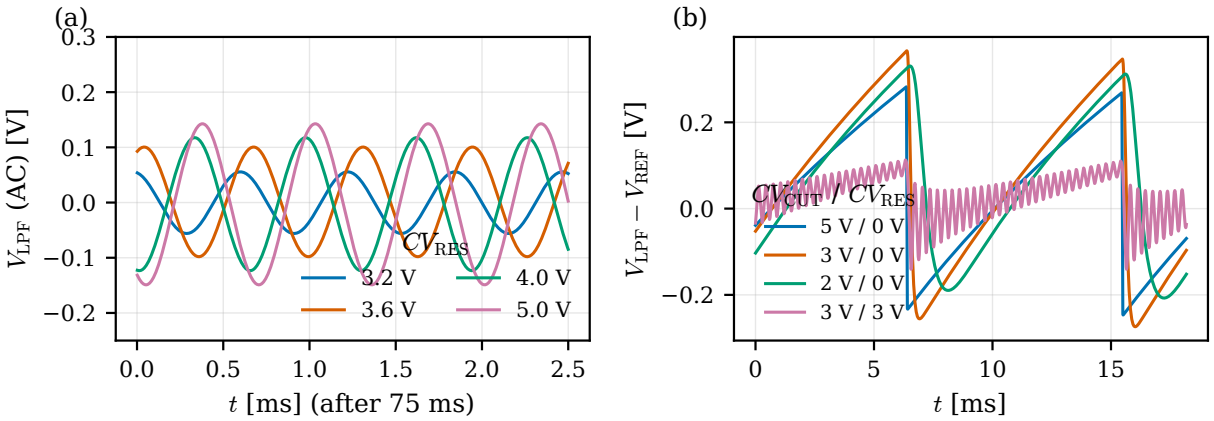


Figure 9: S3: (a) steady-state self-oscillation for several $\mathrm{CV_{RES}}$ above threshold; (b) 110 Hz saw through the filter at different cutoff and resonance settings.

5.9 Design changes found by simulation

Table 7 lists every change that the simulations caused. The most instructive was the *CV network loading* bug:

- **The bug**: the last RC stage of each control-voltage filter was designed as a divider for an ideal source, but it sits behind the series resistance of the first two stages. The real LIN ratio was therefore 0.36 instead of 0.6, and the cutoff range shrank to 44 . . . 4000 Hz.
- **Why the block tests missed it**: the individual block simulations had driven the dividers with ideal voltages.
- **How it was found**: only when the behavioural model (Section 6) could not match the full-voice simulation.
- **Fix**: all networks were redesigned with the loading included, and every simulation now drives the CVs through the real networks.

6 Behavioural model and audio

A fast Python/numba model of the voice (`sim/behav/`) renders audio in seconds instead of the minutes ngspice needs. It mirrors the hardware split:

- **Firmware layer**: patch parameters, envelopes, LFO, glide, accent and slide, timer quantisation, 12-bit CVs at 1 kHz.
- **Analog layer** at 192 kHz: band-limited oscillators (polyBLEP), the S2 filters, a tanh ladder,

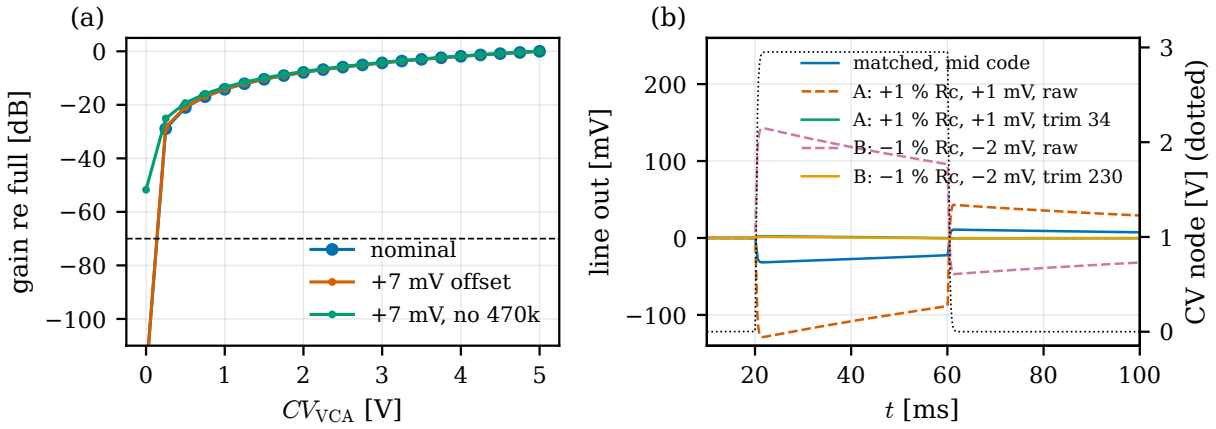


Figure 10: S4: (a) gain versus CV_{VCA} (nominal, with 7 mV op-amp offset, and without the anti-offset resistor); (b) CV feed-through for a 0 → full-scale step without signal, before and after the trim calibration.

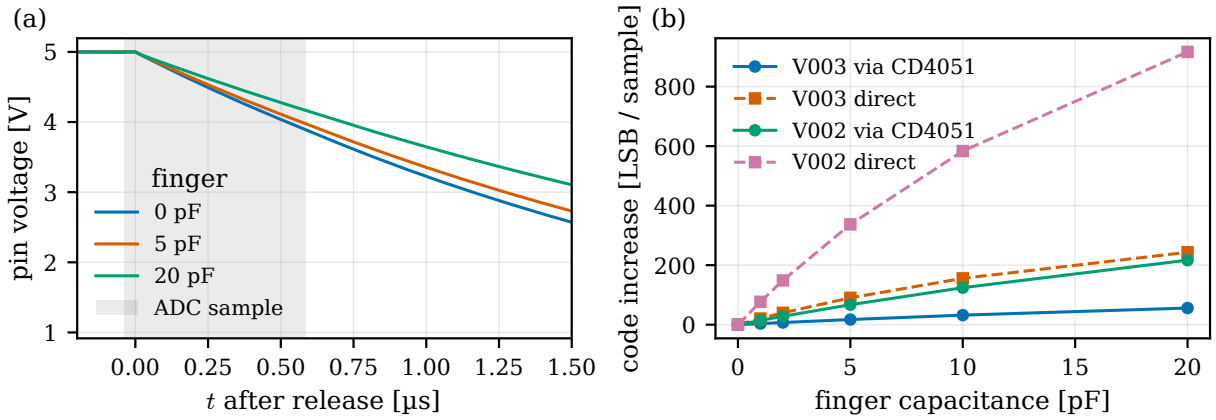


Figure 11: S5: (a) pin voltage after release for 0, 5 and 20 pF finger capacitance; (b) ADC code increase per sample versus finger capacitance, through the multiplexer and direct.

the VCA law and the output coupling.

The model's constants are taken from the ngspice results. The resonance loop gain was obtained by inverting the S3 resonance peaks. Two parameters were fitted to the self-oscillation pitch and amplitude.

Compared with ngspice on the S6 score, the model matches:

- the level within −0.6 dB (1.1 dB rms),
- the self-oscillation pitch within 2 % and its amplitude within 7 %,
- the overall brightness (spectral centroid) within 300 cent; the difference is concentrated where the resonance sits exactly at threshold.

It renders 19 feature clips and three demo tunes in the firmware step format (3 bytes per step): an original acid line, Bach's Prelude in C and Beethoven's *Ode to Joy*. The audio files are in `docs/audio/features/`.

7 PCB design

7.1 Board and stack-up

The board is 100 mm × 100 mm, 1.6 mm, four layers, with all components on the top side (Table 8). A first two-layer attempt was abandoned:

- it could not be routed completely,
- ground islands between dense traces left pads without a reliable ground,

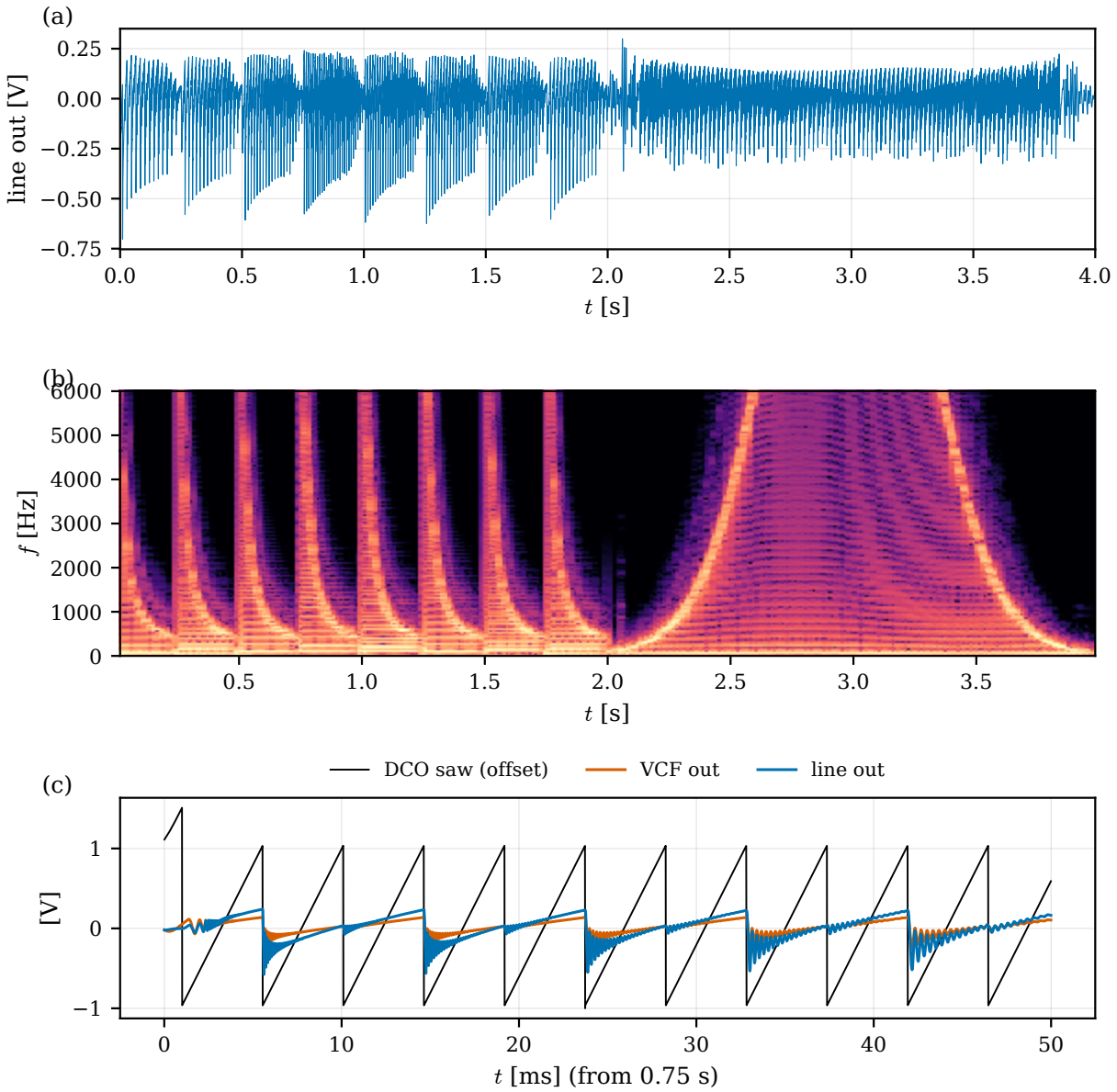


Figure 12: S6: (a) line out; (b) spectrogram; (c) detail at 0.75 s: DCO saw, filter output and line out.

- some touch lines ran under other keys.

The four-layer stack solves all three problems. It also gives the analog audio path a continuous ground reference, at little extra cost.

Touch pads. On the standard JLCPCB stack-up, the first inner layer lies only about 0.2 mm below the pads. A plane there would dominate the pad capacitance and ruin the touch sensitivity. The pads are therefore protected on the other three layers:

- both inner layers have a copper cut-out under every pad,
- the bottom layer has a pour-free keep-out under every pad, open only in a small strip at the top of the pad where the key's own via and trace arrive,
- on the top layer, the pads have a 1 mm local clearance.

7.2 Floorplan

Figure 16 shows the floorplan:

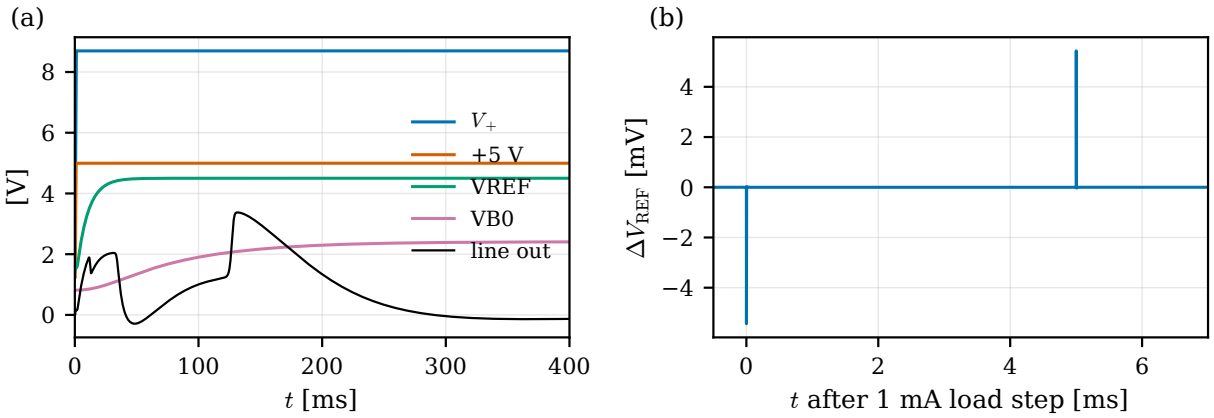


Figure 13: S7: (a) power-up at 9 V; (b) VREF response to a 1 mA load step.

Table 7: Design changes caused by the simulations.

Block	Change	Reason
DCO	2.2 kΩ gate resistor	slower switch-off edge, smaller charge-injection spike
CV filters	10-bit PWM at 46.9 kHz + 2-bit ΣΔ dither	ripple −67 → −81 dB, settling 2.2 → 1.3 ms
CV filters	load-aware networks	correct DC ratios
VCF	ladder capacitors 47 → 33 nF	top of range above 15 kHz
VCF	mixer to VB0 via 10 kΩ / 1 µF, 150 kΩ/470 kΩ sources	settling 0.44 s → 15 ms
VCF	resonance VCA with difference amplifier, 1:13 input, 22 kΩ feedback	no volt-level DC steps into the ladder
VCA	470 kΩ anti-offset resistor	off attenuation 52 → 121 dB
VCA	bases from VREF, 8-bit R-2R trim DAC	feed-through ≈ 100 →< 0.6 mV
VCA	R_E 3 → 4.7 kΩ	maximum gain +11 → +7.4 dB (headroom)
System	audio loopback to PD5	self-test and calibration without extra ICs
Power	VREF follower with 10 Ω + 1 µF snubber	hum −47 → −81 dB, stable

- **Analog area** (top left): the blocks in signal-flow order, from power and references through the oscillator, ladder, output amplifier and resonance to the VCA and the CV filters.
- **Right-hand column**: MIDI input (not populated), the trim DAC directly next to the VCA, and the MCU.
- **Keys I/O row**: the two multiplexers and the two LED shift registers sit in a full-width row directly above the LEDs and keys. Each chip is above the keys it serves, so the 32 LED and touch lines are short vertical drops instead of long buses across the board.

Moving these chips there from a column on the right made the board routable.

7.3 Placement

Placement is scripted (`hw/pcb_place.py`, `hw/gen_pcb.py`):

- **Anchors**: ICs, the ladder transistor column, connectors, keys and LEDs have explicit positions.
- **Greedy placement**: every other part is placed greedily inside its block, most-connected parts first. Each goes to the free spot that minimises the distance to the pads it connects to, with 0.3 mm spacing to leave routing channels.
- **Decoupling capacitors**: they are placed at their IC's supply pin.
- **Test points**: each goes into the block holding most pins of its net, with its net name printed next to it. VCAOUT and XCUT_IN did not fit and sit on the free top edge.

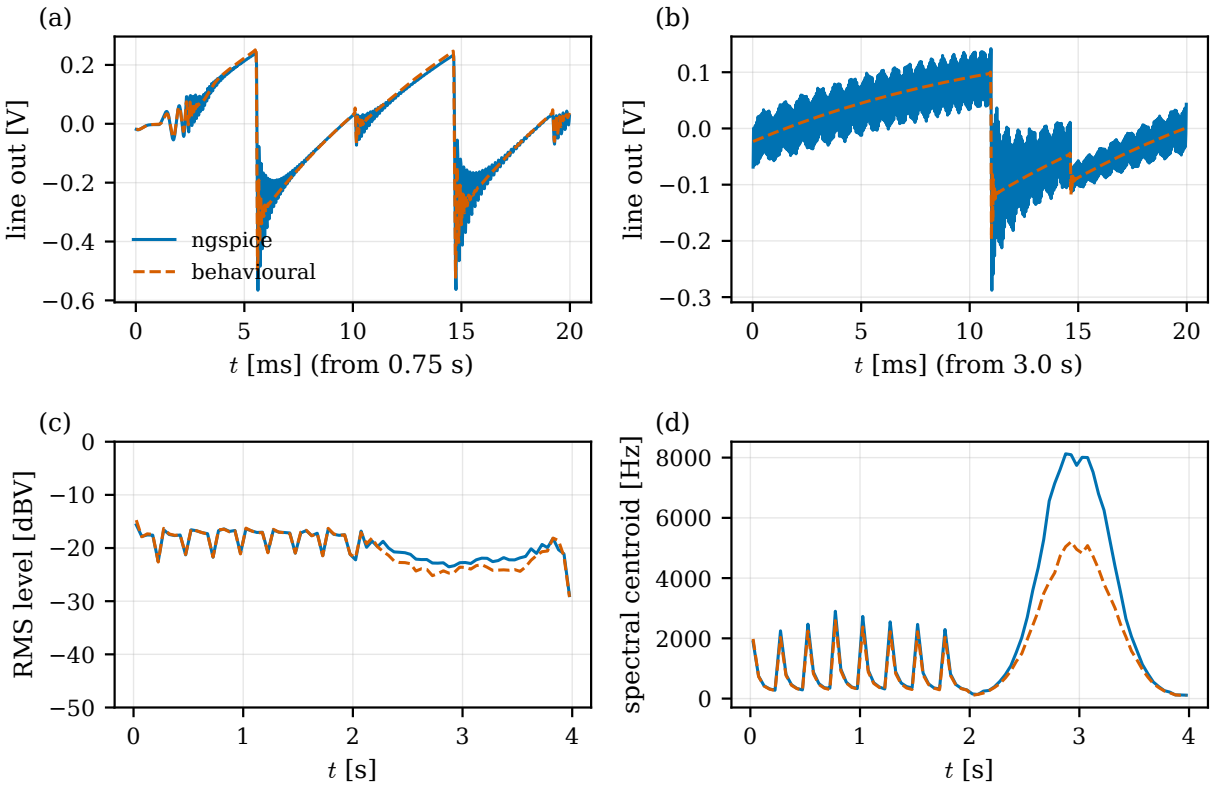


Figure 14: Behavioural model versus ngspice on the S6 score: (a, b) waveform details, (c) RMS level, (d) spectral centroid.

Table 8: Layer stack.

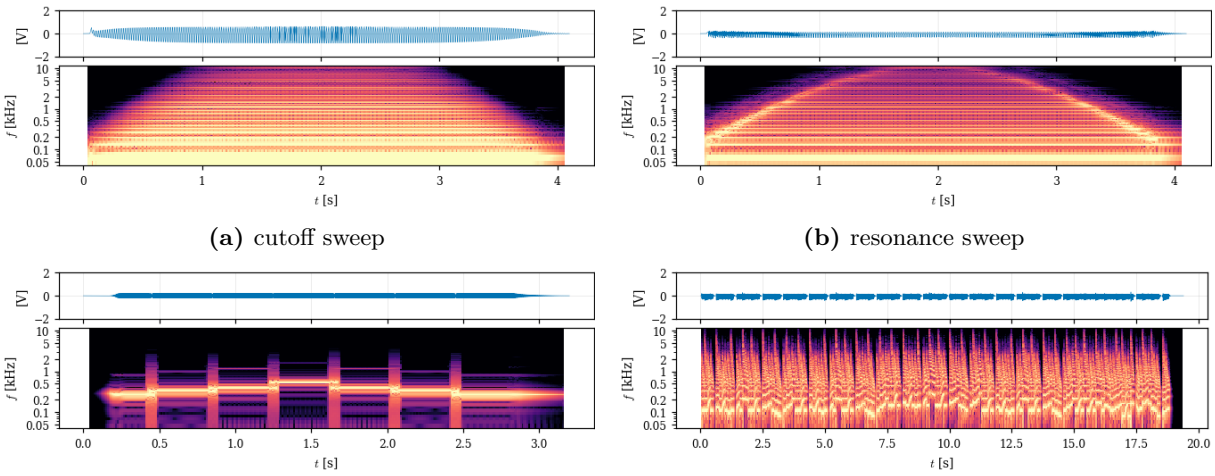
Layer	Use
F.Cu	components, signals, GND pour; no tracks across the keyboard
In1.Cu	solid GND plane
In2.Cu	split power plane: VP under the analog area, +5 V under the MCU column and the keys I/O row, GND under the keyboard
B.Cu	signals, GND pour; below each key only that key's own line

- **Scope ground**: two through-hole 2.54 mm pins on the top edge.

7.4 Routing

Routing is automated in several stages (`gen_pcb.py -route`):

1. **Plane stitching**: every GND, VP and +5 V pad gets a short track to its own via into its plane. The vias are locked so that the autorouter keeps them.
2. **Pre-routing**: the few supply pins outside their plane region are routed on the still empty board with an A* grid router (`hw/pcb_maze.py`) and locked.
3. **Autorouting**: Freerouting 1.9 does two passes. The inner layers are declared as power planes, so signals stay on the outer layers.
4. **Ground grid**: GND stitching vias go on a 2.5 mm grid wherever both outer pours are filled. They keep clear of the mounting holes and of all silkscreen artwork.
5. **Island check**: GND islands are detected by a connectivity analysis and bridged if necessary (`hw/pcb_gnd.py`).
6. **Leftovers**: the A* maze router finishes any remaining connection. A rip-up mode (`-repair`) exists for stubborn cases.



(a) cutoff sweep

(b) resonance sweep

(c) self-oscillation

(d) demo tune (acid line)

Figure 15: Spectrograms of selected behavioural-model clips.

The final board has 0.25 mm tracks (0.3 mm for stitching), 0.2 mm clearance and 0.6 mm/0.3 mm vias, all within JLCPCB's standard capabilities. It has 246 footprints, 1773 track segments (2.5 m on F.Cu, 1.9 m on B.Cu) and 626 vias.

The KiCad design-rule check reports **0 violations, 0 unconnected pads and 0 schematic-parity errors**.

7.5 Silkscreen

The silkscreen documents the design on the board itself:

- a dashed frame with a title for every functional block,
- a piano-style keybed with hatched black keys,
- a strip with the logo, a resonant sawtooth and the ladder response family,
- on the back, the signal-flow diagram and the credits.

All silkscreen text is at least 0.8 mm high with 0.15 mm lines, JLCPCB's minimum.

7.6 Fabrication data

`hw/fab.py` writes the JLCPCB outputs:

- the Gerber files of all four copper layers plus mask, paste, silkscreen and outline, and the Excellon drill file;
- the BOM;
- the placement (CPL) file, with rotation offsets for the SOT, SOIC and TSSOP packages.

The MCU, the headers, the test points, the touch pads and the MIDI parts are excluded from assembly.

8 Bill of materials

JLCPCB assembles 196 parts in 40 BOM lines: 38 Basic and 2 Promotional Extended (BC847BS, CD4051). The components cost about USD 3.65 per board at list price, excluding PCB, assembly fees and the MCU (Table 9). The design has 242 parts in total; 5 are marked do-not-populate (the MCU and the MIDI input).

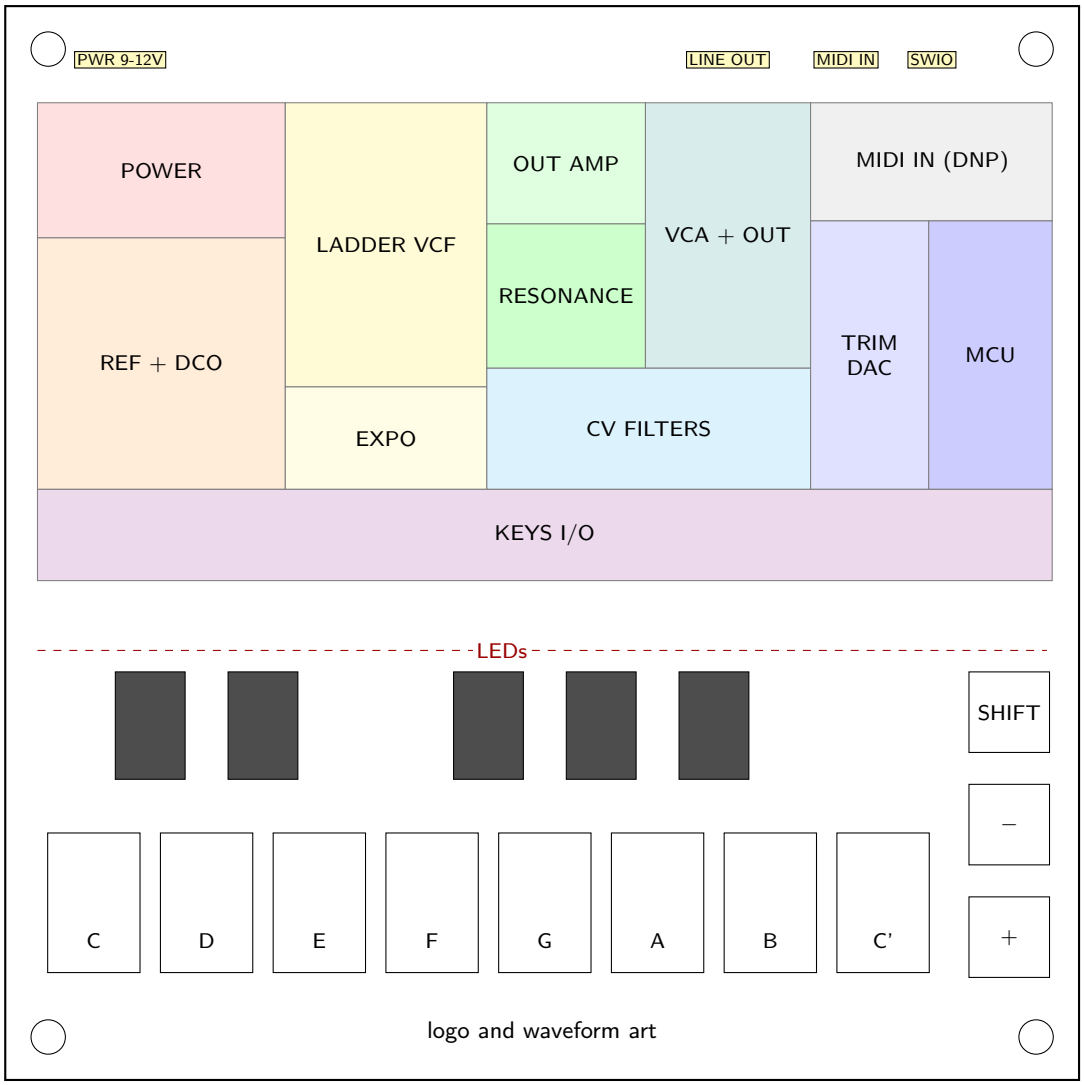


Figure 16: Floorplan (generated from `hw/pcb_place.py`). Connectors along the top edge; the scope ground pins and test points are not shown.

Table 9: Assembled parts (JLCPCB PCBA, top side). Unit prices from the JLCPCB parts list at the time of design.

Value	Qty	Package	LCSC	Library	USD/pc	Designators
4.7k	29	0603	C23162	Basic	0.0028	R16–R31, R49, R50, R56, R58, R75, R76, R83, R84, R91, R92, R95, R102, R103
10k	17	0603	C25804	Basic	0.0018	R6, R7, R55, R63, R64, R66, R67, R73, R81, R82, R85–R88, R104, R106, R109
1k	15	0603	C21190	Basic	0.0026	R1, R9–R12, R48, R52, R57, R62, R74, R78, R90, R94, R98, R99
100k	10	0603	C25803	Basic	0.0031	R34, R36, R38, R40, R42, R44, R46, R65, R68, R108
200k	9	0603	C25811	Basic	0.0023	R32, R33, R35, R37, R39, R41, R43, R45, R47
15k	4	0603	C22809	Basic	0.0029	R51, R77, R93, R110
470k	4	0603	C23178	Basic	0.0030	R70–R72, R96
33k	3	0603	C4216	Basic	0.0022	R53, R59, R97

Value	Qty	Package	LCSC	Library	USD/pc	Designators
22k	3	0603	C31850	Basic	0.0041	R89, R105, R107
1.5k	2	0603	C22843	Basic	0.0019	R5, R60
2.2k	2	0603	C4190	Basic	0.0018	R8, R54
1M	1	0603	C22935	Basic	0.0019	R100
2M	1	0603	C22976	Basic	0.0025	R101
2.4k	1	0603	C22940	Basic	0.0026	R13
9.1k	1	0603	C23260	Basic	0.0031	R2
10	1	0603	C22859	Basic	0.0032	R3
3.3k	1	0603	C22978	Basic	0.0026	R4
300k	1	0603	C23024	Basic	0.0054	R61
150k	1	0603	C22807	Basic	0.0044	R69
6.2k	1	0603	C4260	Basic	0.0023	R79
120k	1	0603	C25808	Basic	0.0015	R80
100n	17	0603	C14663	Basic	0.0122	C2, C4, C6, C15, C17–C22, C26, C28, C34, C36, C41, C45, C48
33n	8	0603	C21117	Basic	0.0106	C23, C27, C30–C33, C37, C42
1µ	8	0603	C15849	Basic	0.0752	C3, C7–C9, C16, C35, C39, C44
10µ	5	0603	C96446	Basic	0.0541	C1, C10–C13
22n	3	0603	C21122	Basic	0.0083	C24, C38, C43
10µ	3	0603	C19702	Basic	0.0319	C5, C14, C46
1n	2	0603	C1588	Basic	0.0089	C29, C47
10n	1	0603	C57112	Basic	0.0108	C25
4.7µ	1	0603	C19666	Basic	0.0294	C40
red	16	0603	C2286	Basic	0.0075	D3–D18
SS14	1	D	C2480	Basic	0.0191	D1
BC847BS	8	SOT-363	C41375126	Promo	0.0281	Q4–Q9, Q11, Q13
MMBT3904	4	SOT-23	C20526	Basic	0.0115	Q1, Q3, Q10, Q12
2N7002	1	SOT-23	C8545	Basic	0.0178	Q2
TL072	3	SOIC-8	C6961	Basic	0.1613	U2, U10, U11
74HC595	3	SOIC-16	C5947	Basic	0.1152	U7–U9
CD4051B	2	SOIC-16	C21379	Promo	0.2574	U5, U6
78L05	1	SOT-89-3	C71136	Basic	0.0892	U1
LM324	1	SOIC-14	C71035	Basic	0.1658	U12
196 parts		40 BOM lines			3.65	per board (parts only)

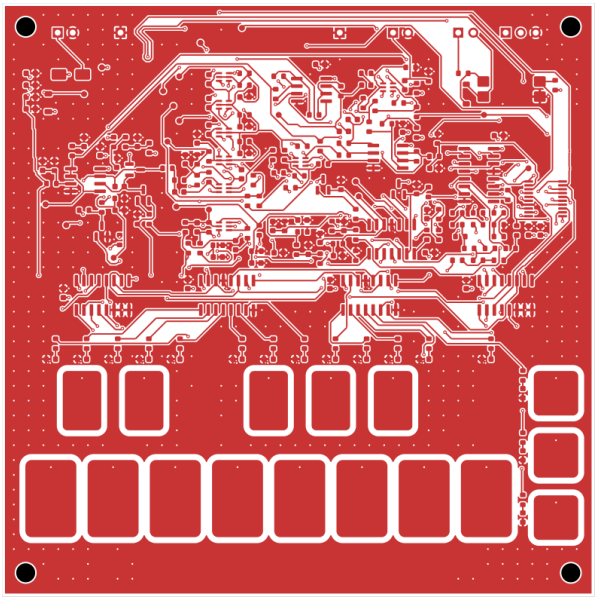
Hand-soldered (not in the assembly BOM):

- the MCU (CH32V003F4P6 or CH32V002F4P6),
- the pin headers: power, line out, SWIO, MIDI and the two scope-ground pins,
- optionally the MIDI parts (LTV-817S, 1N4148W, 220 Ω and 470 Ω).

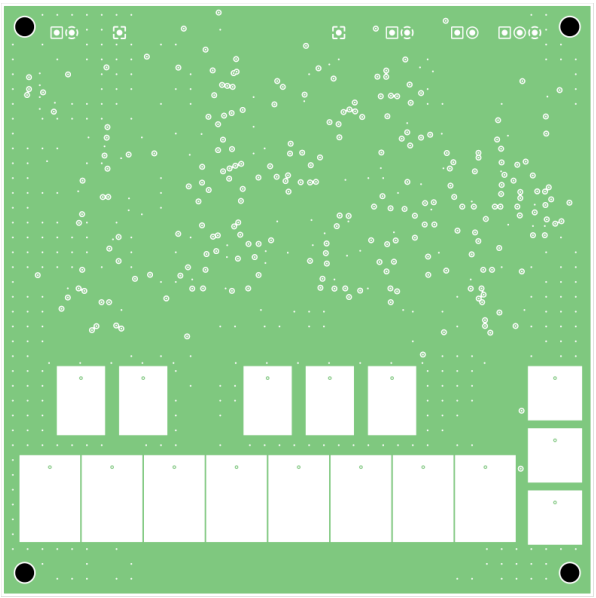
9 Firmware

Firmware version 0.1 (`firmware/touchsynth/`) is built on ch32fun for the CH32V003, CH32V002 and CH32V006 from one code base. It uses 11.7 KB of the 16 KB flash of the smallest part. It has not yet run on hardware. Its structure follows the timing of the design:

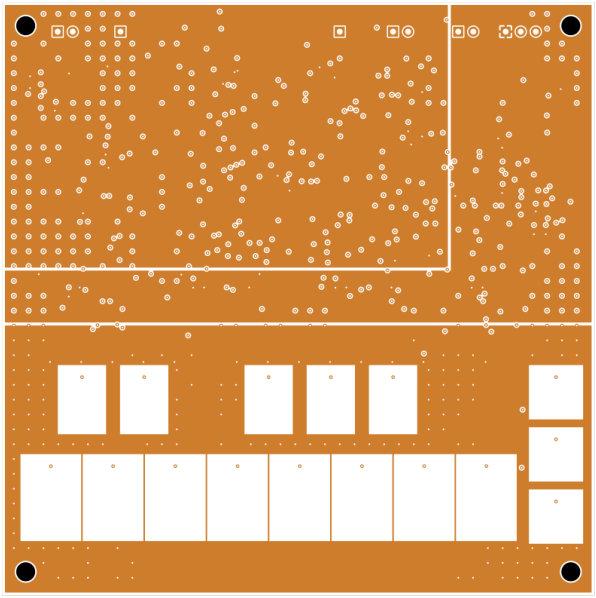
- **TIM1 update interrupt** at 46.875 kHz: loads the four CV compare registers with 2-bit sigma-delta dither (10-bit PWM, 12-bit effective) and clocks the noise LFSR. An interrupt was chosen over DMA bursts for simplicity; it costs a few percent of the CPU.
- **TIM2**: the DCO period. Channel 1 produces the 3 µs active-low reset pulse, channel 2 the pulse wave, and the update interrupt toggles the sub-oscillator.
- **Control loop** at 1 kHz (SysTick): glide, LFO, envelopes, key tracking, the CV laws and the demo-tune player. It mirrors the firmware layer of the behavioural model. All maths is fixed point, because the CH32V003 has no multiplier.



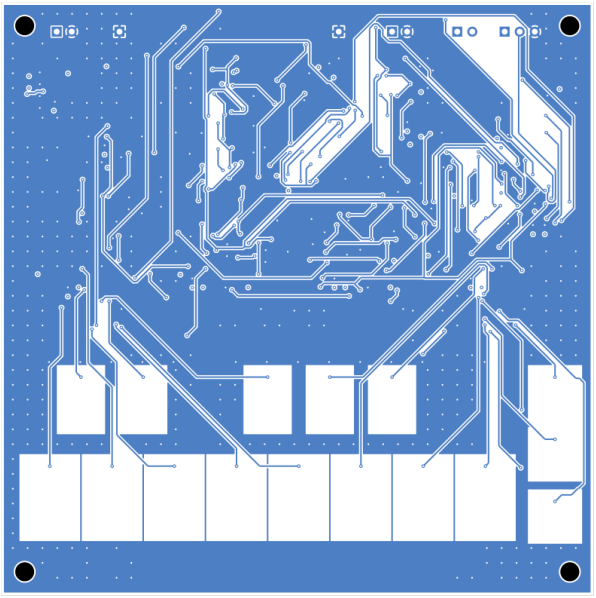
(a) F.Cu



(b) In1.Cu (GND)

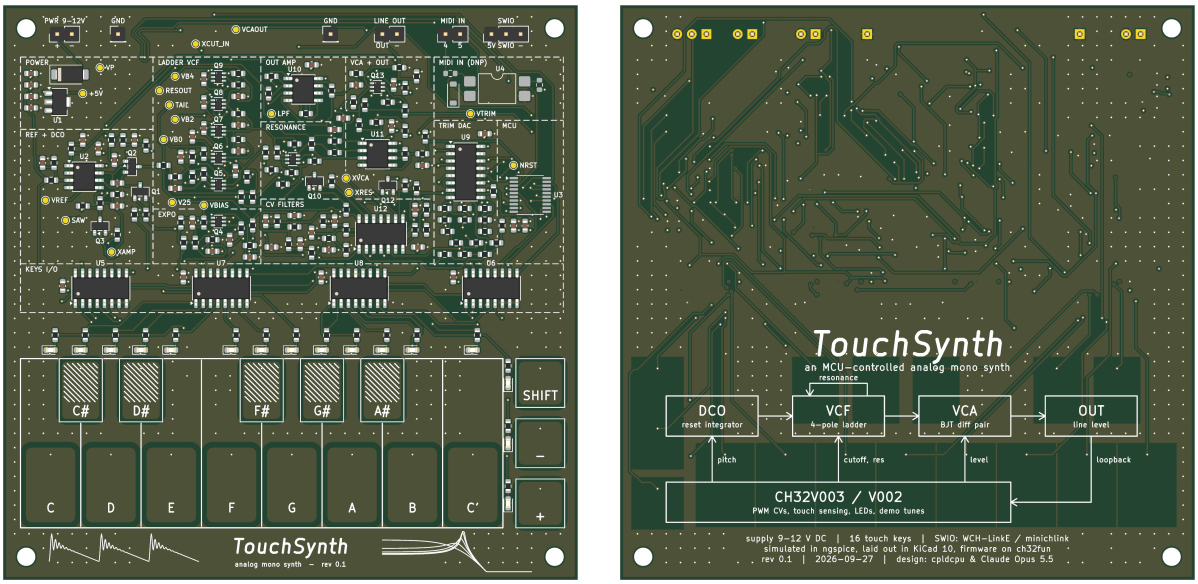


(c) In2.Cu (VP / +5 V / GND)



(d) B.Cu

Figure 17: Copper layers (seen from the top).



(a) top **(b)** bottom

Figure 18: Rendered board.

- **Main loop**: the touch scan of all 16 pads through the multiplexers with per-pad baselines and thresholds, the SHIFT/−/+ user interface, the LED and trim-DAC shift registers, MIDI input, and a test console over SWIO for the verification plan (Section 10).

The calibration tables, presets and demo tunes are generated from the simulation results and `tunes.py` by `gen_tables.py`; nothing is copied by hand.

Pin assignment check. Writing the firmware was also a check of the pin assignment. `check_pins.py` compares every pin constant in the firmware with:

- the net in the design,
- the port in the KiCad symbol,
- the timer, USART and ADC function of that port on each chip family.

All 16 signal pins pass on both families. The check found one difference that the firmware must handle: the timer remap *codes* differ (Table 10). The same remaps also route complementary outputs to PC3, PC4 and PD1 (SWIO). The firmware never enables them.

Table 10: Timer remaps used by the firmware.

Function	CH32V003	CH32V002 / V006
TIM1 CH1–CH4 → PC6, PC7, PC0, PD3	`TIM1_REMAP` = 01	`TIM1_RM` = 0010
TIM2 CH1, CH2 → PC5, PC2	`TIM2_REMAP` = 01	`TIM2_RM` = 010

10 Verification and validation plan

Verification is organised in five levels (Table 11). Levels L2 and L3 are designed to run automatically. A host script drives the test firmware over the WCH-LinkE debug interface with `minichlink`. It captures the audio loopback, computes spectra and compares them with limits derived from the simulations. The results are stored as CSV next to the simulation data, so hardware and simulation can be plotted together.

Table 11: Verification levels.

Level	Content	Method
L0	design verification	ngspice S1–S7 (done)
L1	assembly and bring-up	DMM on the test points, current-limited supply
L2	functional tests T1–T14	test firmware + loopback ADC via `minichlink` (automated)
L3	calibration	per-board constants stored in flash (automated)
L4	validation	playing, listening, long-run stability (user)

10.1 Bring-up

Table 12: Bring-up steps (lab supply 9.0 V, current limit 100 mA).

#	Step	Expected	Limit
B1	visual check, no bridges on the SOIC/TSSOP parts	–	–
B2	reverse polarity, 1 s	$I < 1$ mA	pass
B3	input current, MCU not fitted	≈ 10 mA	5 … 25 mA
B4	V+	8.6 V	8.4 … 8.8 V
B5	+5 V	5.0 V	4.8 … 5.2 V
B6	VREF	4.50 V	4.45 … 4.55 V
B7	VB0 … VB4	2.40 / 3.40 / 4.40 / 5.40 / 6.40 V	±3 %
B8	TAIL	1.86 V	1.7 … 2.0 V
B9	LPF (no signal)	4.50 V	4.3 … 4.7 V
B10	VCAOUT	4.50 V	4.3 … 4.7 V
B11	fit MCU, flash test firmware, `id` answers	–	–
B12	input current with MCU, LEDs off	+5 … 8 mA	< 40 mA

10.2 Functional tests

Table 13: Functional tests (automated `selftest`), limits derived from the simulations.

ID	Block	Procedure	Pass limit (sim)
T1	LEDs, 595 chain	walking one; trim DAC walk shifts VCAOUT DC monotonically	monotonic, > 20 mV span
T2	touch	256 readings per pad untouched; user touches each pad	$\sigma < 2$ LSB (V003) / 6 LSB (V002); $\Delta > 10\sigma$
T3	touch scan	time a full 16-pad scan	≥ 200 Hz (sim 1.4 kHz)
T4	DCO pitch	loopback capture at C2, C4, C6	< 1 cent
T5	DCO amplitude	loopback amplitude C1–C8	±5 % after calibration
T6	VCF self-osc. threshold	ramp CV_{RES}, detect sustained oscillation	2.7 … 3.5 V (sim 3.1 V)
T7	VCF tracking	self-oscillation frequency at CV_{CUT} = 1–4.5 V	2.05 ± 0.2 oct/V
T8	VCF low-pass	110 Hz saw, cutoff 1 kHz, harmonic 10 vs. 1	±3 dB of sim
T9	VCA gain law	1 kHz saw, CV_{VCA} sweep	max. gain ±2 dB of sim (+7.4 dB)

ID	Block	Procedure	Pass limit (sim)
T10	VCA thump	CV_{VCA} steps without signal, before/after calibration	$< 3\,mV$ after (sim 0.6 mV)
T11	CV settling	VCA step seen at the loopback	1 % in $< 2.5\,ms$ (sim 1.3 ms)
T12	hum and noise	VCA open, DCO off, spectrum	100 Hz < sim + 6 dB; broadband $< -60\,dB$
T14	MIDI (if fitted)	note on/off, pitch bend, CC from a keyboard	all received

10.3 Calibration and validation

- **Calibration (L3)**: the calibration stores the DCO gain (expected 0.95–1.05), the cutoff scale and offset (from the self-oscillation frequency at two CV codes), the VCA trim code (expected 30–230) and per-pad touch baselines and thresholds.
- **Validation (L4)**: covers reliable playing without ghost or double triggers, latency below 10 ms, tuning stability, usability of the parameter UI, the sound compared with the behavioural-model examples, the demo tunes, hum with typical adapters, and MIDI.

Table 14: Traceability from requirements to simulation and hardware tests.

Requirement	Simulation	Hardware test
DCO range and amplitude ±5 %	S1	T4, T5
CV ripple and settling	S2	T11
VCF range, slope, resonance	S3	T6, T7, T8
VCA attenuation, thump	S4	T9, T10
Touch sensitivity and scan rate	S5	T2, T3, V1, V2
Complete voice	S6	V5
Power, hum	S7	B2–B5, T12, V6

11 Open points and risks

- **Accepted limitations.**
  - Power-on pop at the line output (about 3 V, 0.3 s).
  - Filter cutoff temperature drift of about −13 cent/K. It could be compensated in firmware by re-tuning from the self-oscillation frequency, measured through the loopback.
- **Not modelled in simulation.**
  - The TL072 input common-mode limit. The design keeps all TL072 inputs at or above 4 V.
  - Noise.
  - Transistor mismatch beyond the explicit cases.
  - PCB parasitics, including key-to-key coupling of the touch pads and switching noise from the LEDs and shift registers. These are covered by T2 and T12.
- **Touch sensitivity on the 4-layer board.** The inner planes are cut out under the pads, but the touch lines run over the power plane and have more capacitance than assumed in S5. Check with T2; if needed, increase the number of samples or lower the thresholds.
- **CH32V006 pinout.** The firmware builds for the CH32V006 (more flash and RAM). Its TSSOP-20 pinout is assumed identical to the CH32V002; check it against the V006 datasheet before fitting one.
- **Assembly.** Check the orientation of the SOT-23/SOT-363 transistors, the diode and the LEDs at the first inspection. Hand-solder the TSSOP-20 MCU.

- **Documentation.** The specification (`docs/spec.md`) and the verification plan still describe some pre-layout details (two layers, test pads on the bottom side). This report reflects the design as ordered.

A Schematic

The following pages reproduce the KiCad schematic (`docs/touchsynth_schematic.pdf`) scaled to A4. The PDF is vector graphics and can be zoomed.

TouchSynth – MCU-controlled analog mono synth (CH32V003/V002)

power

Power, references and ladder bias

Datei: power.kicad_sch

mcu

MCU, programming header and MIDI in

Datei: mcu.kicad_sch

touch_led

Touch pads, multiplexers, LEDs and trim DAC

Datei: touch_led.kicad_sch

vcf

CV_CUT, expo converter, ladder filter, mixer

Datei: vcf.kicad_sch

resonance

CV_RES and resonance VCA

Datei: resonance.kicad_sch

vca

CV_VCA, VCA, line out and audio loopback

Datei: vca.kicad_sch

Sheet: /
File: touchsynth.kicad_sch

Title: TouchSynth

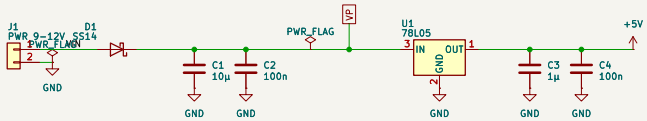
Size: A4	Date: 2026-09-27	**Rev: 0.1**
KiCad E.D.A. 10.0.6		Id: 1/8

TouchSynth — design report

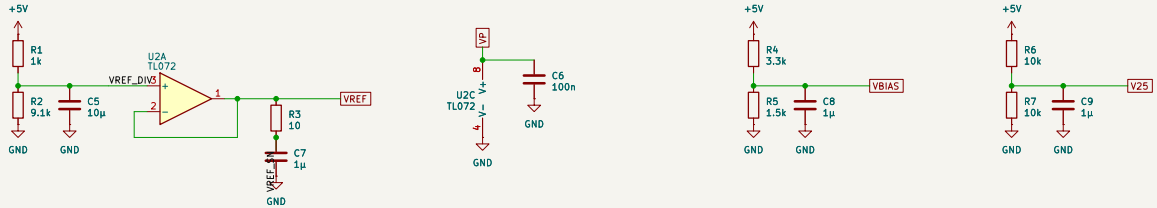
rev 0.1

Power, references and ladder bias

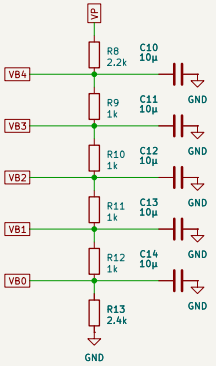
Input, reverse protection, 5 V



References: VREF 4.5 V (buffered), VBIAS 1.5 V, V25 2.5 V



Ladder bias chain (from V+; VB0…VB4 = 2.4…6.4 V at V+ = 8.6 V)



Test points



TouchSynth — design report

rev 0.1

Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

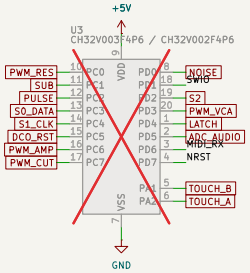
Sheet: /power/
File: power.kicad_sch

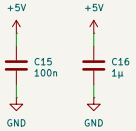
Title: TouchSynth — Power, references and ladder bias

Size: A3	Date: 2026-09-27	Rev: 0.1
KiCad E.D.A. 10.0.6		Id: 2/8

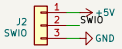
MCU, programming header and MIDI in

MCU: CH32V003F4P6 or CH32V002F4P6 (TSSOP-20, hand-soldered, not placed by JLCPCB)



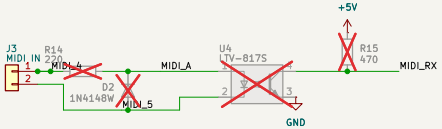


Programming (WCH-LinkE, SWIO)





MIDI in (not populated in rev 0.1)



TouchSynth — design report

rev 0.1

27

Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

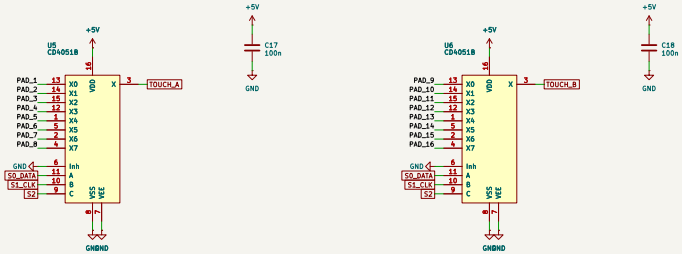
Sheet: /mcu/
File: mcu.kicad_sch

Title: TouchSynth — MCU, programming header and MIDI in

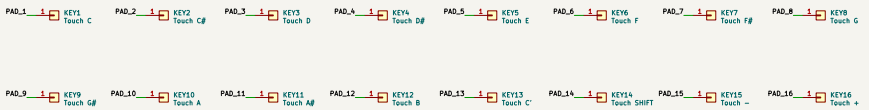
Size: A3	Date: 2026-09-27	Rev: 0.1
KiCad E.D.A. 10.0.6		Id: 3/8

Touch pads, multiplexers, LEDs and trim DAC

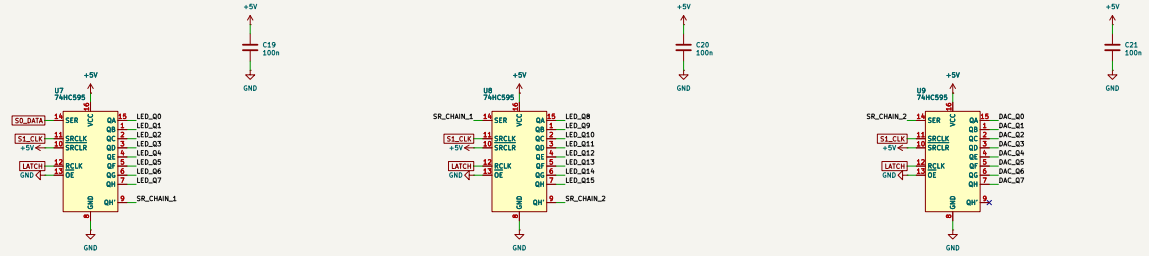
Touch multiplexers (select S0…S2 shared with the 74HC595 data / clock)



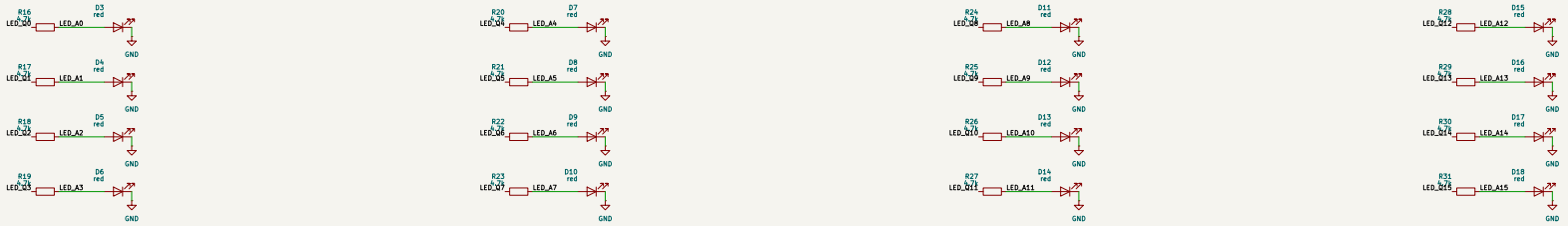
Touch pads (copper under solder mask)



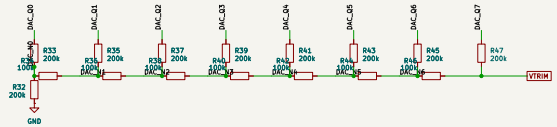
Shift registers (daisy-chained: LEDs 0–7, LEDs 8–15, trim DAC)



LEDs (4.7k series resistors, ~0.6 mA each)



VCA trim DAC: 8-bit R-2R ladder (R = 100k, 2R = 200k), output impedance 100k



TouchSynth — design report

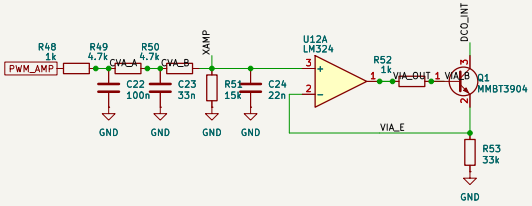
rev 0.1

Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

Sheet: /touch_led/	
File: touch_led.kicad_sch	
Title: TouchSynth — Touch pads, multiplexers, LEDs and trim DAC	
Size: A2	Date: 2026-09-27
KiCad E.D.A. 10.0.6	Id: 4/8
	Rev: 0.1

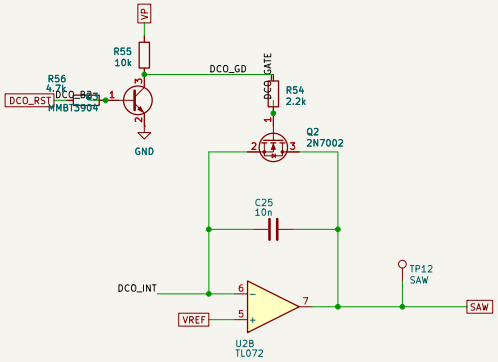
DCO and CV_AMP

CV_AMP: PWM filter and V−to−I converter (charge current proportional to the note frequency)





Integrator (TL072) with 2N7002 reset and NPN gate level shifter



Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

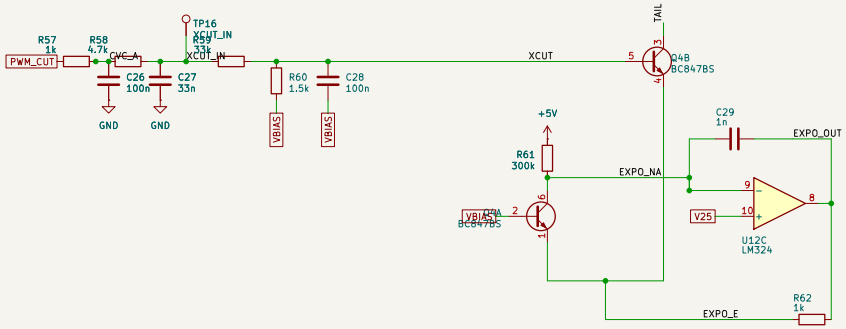
Sheet: /dco/
File: dco.kicad_sch

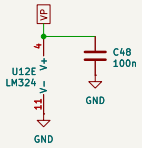
Title: TouchSynth — DCO and CV_AMP

Size: A3	Date: 2026-09-27	Rev: 0.1
KiCad E.D.A. 10.0.6		Id: 5/8

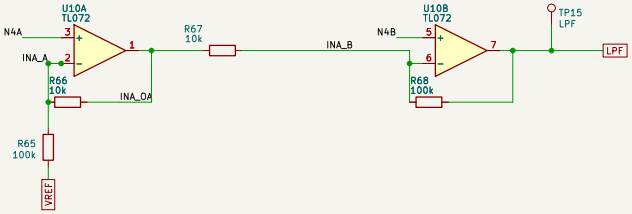
CV_CUT, expo converter, ladder filter, mixer

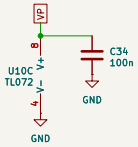
CV_CUT: PWM filter and exponential converter (tail current of the ladder)



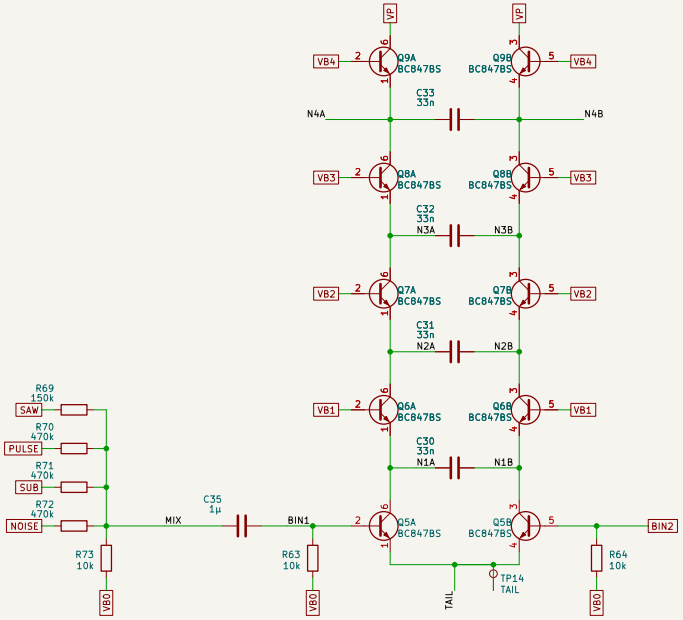


Output instrumentation amplifier across the top capacitor (G = 11, referenced to VREF)





Transistor ladder (4 poles, 4 x 33 nF), bias from VB0...VB4, mixer



Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

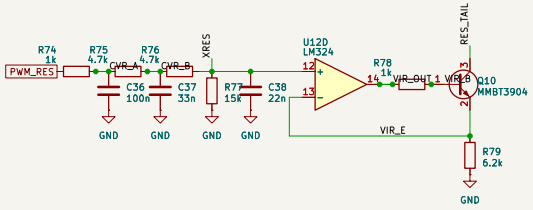
Sheet: /vcf/	
File: vcf.kicad_sch	
Title: TouchSynth — CV_CUT, expo converter, ladder filter, mixer	
Size: A3	Date: 2026-09-27
KiCad E.D.A. 10.0.6	Id: 6/8

Rev: 0.1

rev 0.1

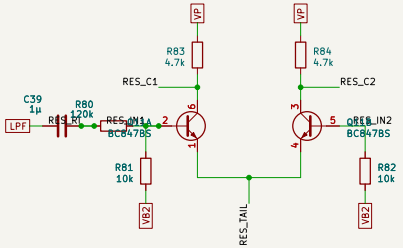
CV_RES and resonance VCA

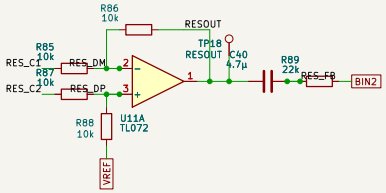
CV_RES: PWM filter and V−to−I converter (tail current of the resonance VCA)





Resonance VCA: differential pair, difference amplifier, AC−coupled feedback into the ladder (BIN2)





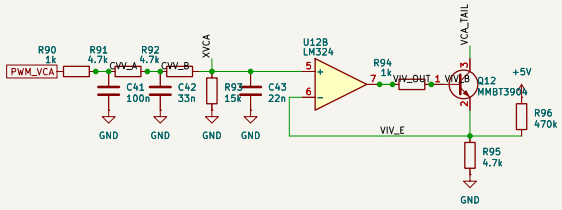
Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

Sheet: /resonance/	
File: resonance.kicad_sch	
Title: TouchSynth — CV_RES and resonance VCA	
Size: A3	Date: 2026-09-27
KiCad E.D.A. 10.0.6	Id: 7/8

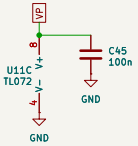
Rev: 0.1

CV_VCA, VCA, line out and audio loopback

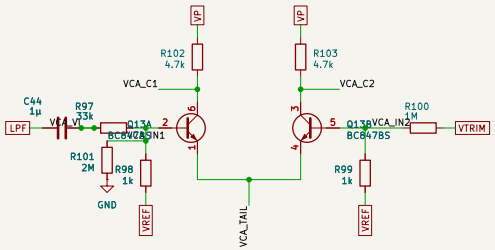
CV_VCA: PWM filter and V−to−I converter (470k pull−up guarantees zero current at CV = 0)

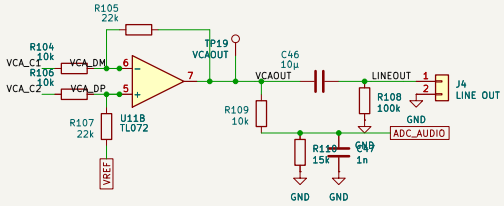






VCA: differential pair with offset trim, difference amplifier, line out, audio loopback





Generated by hw/gen_sch2.py (layouts.py) from hw/design.py

Sheet: /vca/
File: vca.kicad_sch

Title: TouchSynth — CV_VCA, VCA, line out and audio loopback

Size: A3	Date: 2026-09-27	Rev: 0.1
KiCad E.D.A. 10.0.6		Id: 8/8

B Reproducing the design

```
cd sim/s1_dco && python s1_dco.py      # likewise s2_cv ... s7_power (order: S1, S3, S4, S6)
cd sim/behav && python validate.py && python examples.py
cd hw && python gen_sch2.py && python check_netlist.py && python check_sim.py
cd hw && <KiCad python> gen_pcb.py --route  # placement, routing, fill, stitching
cd hw && python fab.py                      # gerbers, BOM, CPL
cd docs/report && python gen_tables.py && latexmk -pdf touchsynth_report.tex
```